

山东大学 网络空间安全学院

创新创业实践课 第一次实验作业二

组员

李瑞佳

岳智宇

薛俊东

张金浩

学

院

网络空间安全学院

完成时间

2026 年 7 月 20 日

目录

一 引言	1
二 ECDSA 密码算法基础	1
2.1 secp256k1 曲线参数	1
2.2 ECDSA 签名与验证的数学流程	2
2.2.1 密钥生成	2
2.2.2 签名生成 (secp256k1_ecdsa_sig_sign)	2
2.2.3 签名验证 (secp256k1_ecdsa_sig_verify)	3
2.3 跳过消息哈希时的签名伪造攻击	4
三 密码算法相关的错误修复	5
3.1 ECDSA 签名 $r = 0$ 边界条件错误	5
3.1.1 问题描述	5
3.1.2 修复方案	6
3.1.3 数学分析	6
3.2 Nonce 重用攻击与确定性签名的根本重要性	7
3.2.1 Nonce 重用的代数攻击	7
3.2.2 RFC 6979 的确定性方案	7
3.3 签名可锻性与 Low-S 规范化	8
3.3.1 可锻性的数学根源	8
3.3.2 比特币中的影响	8
3.3.3 libsecp256k1 中的实现	8
四 密码算法相关的性能优化	9
4.1 safegcd 模逆算法	9
4.1.1 为什么模逆运算是性能瓶颈	9

4.1.2	safegcd 算法的核心创新	9
4.1.3	收敛性： δ 的作用与迭代界	10
4.2	SDMC 带符号位多梳点乘算法	11
4.2.1	算法配置	11
4.2.2	标量重编码与数学推导	11
4.2.3	盲化与安全设计	12
4.2.4	源码中的时间侧信道防护细节	12
4.3	128 位宽乘法与 Montgomery 模乘	12
4.3.1	数学需求：为什么需要 128 位乘法	13
4.3.2	编译器差异与 MSVC 的挑战	13
4.3.3	修复方案：MSVC 内置函数	13
4.3.4	性能提升	14
4.4	ECDH 点乘算法替换与 x86_64 汇编移除	14
4.4.1	ECDH 点乘算法的替换	15
4.4.2	启发	15
五	数学原理总结	16
5.1	椭圆曲线群运算	16
5.2	椭圆曲线离散对数问题	16
5.3	模逆与 GCD 算法的关系	17
5.4	标量乘法的代数优化空间	17
5.5	哈希函数在 ECDSA 中的不可替代性	18

一 引言

比特币是全球最大的加密货币系统，其核心安全基础建立在椭圆曲线数字签名算法 ECDSA 之上。比特币使用 secp256k1 曲线——一条由 SECG 标准化的椭圆曲线。所有比特币交易的合法性均通过 ECDSA 签名来保证：只有拥有对应私钥的人才能花费其比特币。

bitcoin-core/secp256k1 是 Bitcoin Core 维护的高度优化的 C 语言椭圆曲线密码库。该项目的目标是在 secp256k1 曲线上提供最高质量的数字签名和密码原语实现。其核心设计原则包括：

- **常数时间实现**：所有涉及秘密数据的代码路径均以常数时间运行，不依赖数据值
- **零动态内存分配**：签名和验证操作不调用 malloc，杜绝内存分配相关的侧信道
- **RFC 6979 确定性签名**：默认使用确定性的 nonce 生成，从根本上消除随机数缺陷风险
- **强验证**：默认拒绝非规范签名（高 S 值签名和宽松 DER 编码）

二 ECDSA 密码算法基础

2.1 secp256k1 曲线参数

secp256k1 是定义在有限素数域 $\mathbb{F}_p$ 上的 Koblitz 椭圆曲线，方程为 Weierstrass 标准形式 $y^2 = x^3 + ax + b$ 的特例 ($a = 0, b = 7$):

曲线方程（仿射坐标）：

$$y^2 = x^3 + 7$$

基域素数 p （256 位）：

$$p = 2^{256} - 2^{32} - 977$$

在源码 `src/ecdsa_impl.h` 中以 8 个 32 位 limb 的形式定义。

曲线阶 n (即点的数量, 256 位素数):

```
n = 0xFFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFE
      BAAEDCE6 AF48A03B BFD25E8C D0364141
```

在 `src/scalar_4x64_impl.h` 中定义 limb 常量 `SECP256K1_N_0` 到 `SECP256K1_N_3`。

余因子 $h = 1$: 曲线阶本身就是素数, 群为循环群。

生成元 G :

```
Gx = 0x79BE667E F9DCBBAC 55A06295 CE870B07
      029BFCDB 2DCE28D9 59F2815B 16F81798
```

p 与 n 的关系: $2n > p > n$ 。这意味着对于验证中的 x 坐标比较 ($x \bmod n$), 最多只有两种可能 ($h \in \{0, 1\}$), 这是 `ecdsa_sig_verify` 中高效验证的关键数学事实。源码中 `secp256k1_ecdsa_const_p_minus_order` 预计算了 $p - n$ 的值。

$a = 0$ 的意义: 仿射坐标下的点加倍公式为 $\lambda = \frac{3x^2+a}{2y}$ 。当 $a = 0$ 时分子简化为 $3x^2$, 减少了一次域加法。此外, `secp256k1` 还具有高效可计算的自同态 ϕ (满足 $\phi^2 + \phi + 1 = 0$), 在验证路径的点乘中被用于 GLV 分解, 将 256 位标量拆分为两个 128 位标量。

2.2 ECDSA 签名与验证的数学流程

2.2.1 密钥生成

私钥 d 是 $[1, n - 1]$ 内的随机整数。公钥 $P = d \cdot G$ 通过标量乘法计算。

2.2.2 签名生成 (`secp256k1_ecdsa_sig_sign`)

对消息 m 生成签名 $\sigma = (r, s)$ 的完整数学过程 (对应源码第 274–307 行):

Step 1: 选择 nonce $k \in [1, n - 1]$;

Step 2: 计算 $R = k \cdot G = (R_x, R_y)$;

Step 3: 取 $r = R_x \bmod n$;

Step 4: 计算 $e = \text{SHA-256}(m)$;

Step 5: 计算 $s = k^{-1} \cdot (e + d \cdot r) \bmod n$;

Step 6: 对 s 执行 Low-S 规范化;

Step 7: 若 $r = 0$ 或 $s = 0$, 返回失败让调用者重试。

2.2.3 签名验证 (secp256k1_ecdsa_sig_verify)

验证过程 (源码第 195–271 行):

Step 1: 检查 $r \neq 0$ 且 $s \neq 0$;

Step 2: $w = s^{-1} \bmod n$;

Step 3: $u_1 = e \cdot w \bmod n$, $u_2 = r \cdot w \bmod n$;

Step 4: $R' = u_1 \cdot G + u_2 \cdot P$;

Step 5: 验证 $x(R') \bmod n = r$ 。

验证的数学正确性证明:

$$\begin{aligned} R' &= e \cdot s^{-1} \cdot G + r \cdot s^{-1} \cdot P \\ &= s^{-1}(e \cdot G + r \cdot P) \\ &= s^{-1}(e \cdot G + r \cdot d \cdot G) \\ &= s^{-1}(e + d \cdot r) \cdot G \\ &= [k^{-1}(e + d \cdot r)]^{-1} \cdot (e + d \cdot r) \cdot G \\ &= k \cdot G = R \end{aligned} \tag{1}$$

验证中的关键优化 (源码第 241–270 行):

验证时需要检查计算出的点 R' 的 x 坐标模 n 后是否等于 r 。但在 Jacobian 坐标下, $x = X/Z^2 \bmod p$, 提取 x 需要计算 Z^2 的模逆——这是域 $\mathbb{F}_p$ 上的求逆, 代价很高。

源码利用 $2n > p > n$ 这个事实绕过了模逆。设签名中的 r 已知, 要验证 $(X/Z^2 \bmod p) \bmod n = r$, 即 $X/Z^2 \equiv r \pmod{n}$ 。由于 $X/Z^2 \in [0, p-1]$ 且 $n < p < 2n$, 最多只

有两种可能：

$$X/Z^2 = r \quad \text{或} \quad X/Z^2 = r + n \quad (\text{且 } r + n < p)$$

两边乘以 Z^2 ，转化为无需模逆的等价条件：

$$X \equiv r \cdot Z^2 \pmod{p} \quad \text{或} \quad X \equiv (r + n) \cdot Z^2 \pmod{p} \quad (\text{且 } r + n < p)$$

源码中的 `secp256k1_gej_eq_x_var` 先测试第一种情况，若失败且 $r + n < p$ 再测试第二种。整个过程只用模乘，完全消除了模 p 逆。

2.3 跳过消息哈希时的签名伪造攻击

ECDSA 验证中有一个看似冗余但至关重要的约束：验证方必须自行计算 $e = \text{Hash}(m)$ ，而不能接受外部传入的哈希值。若跳过此步骤，任何人都可以在不知道私钥 d 的情况下为任意公钥 P 伪造有效签名。

通用伪造构造：

选择任意 $u, v \in \mathbb{Z}_n^*$ ，计算：

$$R' = u \cdot G + v \cdot P = (x', y')$$

令 $r' = x' \bmod n$ ， $s' = r' \cdot v^{-1} \bmod n$ ， $e' = r' \cdot u \cdot v^{-1} \bmod n$ 。

则 (r', s') 对消息哈希 e' 构成有效签名，因为：

$$s'^{-1}(e' \cdot G + r' \cdot P) = v \cdot r'^{-1} \cdot (r' \cdot u \cdot v^{-1} \cdot G + r' \cdot P) = u \cdot G + v \cdot P = R'$$

该攻击为何在现实中不可行：

在 SHA-256 的安全性假设下，攻击者无法找到消息 m' 使得 $\text{SHA-256}(m') = e'$ （抗原像性）。这一分析揭示了 ECDSA 的安全性是 ECDLP 困难性和哈希函数不可逆性的联合保证，两者必须同时成立，ECDSA 才是安全的。

三 密码算法相关的错误修复

3.1 ECDSA 签名 $r = 0$ 边界条件错误

3.1.1 问题描述

1. 根据 ECDSA 协议规范，签名过程中计算得到

$$r = R_x \bmod n,$$

若 $r = 0$ ，则签名者必须重新选择随机数 k 并重新生成签名。这是因为合法的 ECDSA 签名要求 $r, s \in [1, n - 1]$ ，当 $r = 0$ 时，生成的签名无法通过验证。

2. 2016 年，Bitcoin Core 在 commit25e3cbbf 中对函数 `ecdsa_sig_sign` 进行了优化，将原本用于检查 r 和 s 是否为零的运行时检查替换为 `VERIFY_CHECK`。开发者认为，这些检查验证的是输入随机数 k 是否为零，而调用方已经保证 $k \neq 0$ ，因此这些判断永远不会失败，可以视为冗余代码。

3. 然而，这一推断混淆了随机数 k 与签名分量 r 的关系。实际上，检查的对象并非输入的 k ，而是由

$$R = kG$$

计算得到的椭圆曲线点 R 的横坐标满足

$$r = R_x \bmod n.$$

虽然调用方能够保证 k 为合法的非零值，但无法预先判断计算得到的 r 是否恰好等于零。因此，该检查并非不可达代码，而是 ECDSA 协议要求保留的运行时验证。

4. 在 Debug 构建中，`VERIFY_CHECK` 会触发断言，从而终止程序；而在 Release 构建中，该宏会被完全移除。当极少数情况下出现 $r = 0$ 时，函数不会返回错误并重新生成随机数，而是继续生成包含 $r = 0$ 的无效签名，违反了 ECDSA 协议规范。这一错误最终形成了 CVE-2016-10150 漏洞。

3.1.2 修复方案

2020 年, Tim Ruffing 提交 PR #732 对该漏洞进行了修复, 主要包括以下两个方面:

1. 回退 commit 25e3cfbf 引入的修改, 将 VERIFY_CHECK 恢复为运行时检查 runtime check, 确保当签名分量 r 或 s 为零时函数能够返回错误, 而不是继续生成无效签名。
2. 在保证常数时间执行的前提下, 将返回值改写为按位运算形式。
3. 上述实现分别判断 r 和 s 是否为零。当二者均非零时返回 1; 若任一值为零, 则返回 0, 通知调用者重新生成随机数并重新执行签名过程, 从而保证生成的签名满足 ECDSA 协议规范。

3.1.3 数学分析

在 ECDSA 中, 签名分量定义为

$$r = R_x \bmod n,$$

其中 $R = kG$, n 为基点 G 的阶。当且仅当

$$R_x \equiv 0 \pmod{n}$$

时, 有 $r = 0$ 。对于 secp256k1 曲线, 由于 R_x 在模 n 意义下近似均匀分布, 因此发生该事件的概率约为

$$\Pr[r = 0] \approx \frac{1}{n} \approx 2^{-256} \approx 10^{-77},$$

几乎不可能在正常运行过程中自然出现。

尽管如此, ECDSA 协议仍要求对这一极端边界情况进行处理。当检测到 $r = 0$ (或 $s = 0$) 时, 签名算法必须放弃当前随机数 k , 重新生成新的 nonce 并重新计算签名。

3.2 Nonce 重用攻击与确定性签名的根本重要性

虽然不是 libsecp256k1 代码库中的 bug，但 RFC 6979 确定性签名的引入是对 ECDSA 历史上最严重错误模式的系统性修复：nonce 重用。

3.2.1 Nonce 重用的代数攻击

设同一私钥 d 对两个消息 m_1 和 m_2 使用了相同的 nonce k （两个签名有相同的 r ）：

$$\begin{cases} s_1 = k^{-1}(e_1 + d \cdot r) \bmod n \\ s_2 = k^{-1}(e_2 + d \cdot r) \bmod n \end{cases} \quad (2)$$

两式相减消去私钥项：

$$s_1 - s_2 = k^{-1}(e_1 - e_2) \bmod n$$

$\Downarrow$

$$k = (e_1 - e_2) \cdot (s_1 - s_2)^{-1} \bmod n$$

恢复 k 后代入任一签名方程即可解出私钥：

$$d = r^{-1} \cdot (s_1 \cdot k - e_1) \bmod n$$

3.2.2 RFC 6979 的确定性方案

RFC 6979 的核心思想： k 不由外部随机数生成器产生，而是通过 HMAC-SHA256 从私钥 d 和消息哈希 e 中确定性地派生：

$$k = \text{HMAC_DRBG}(d \parallel e)$$

这确保同一 (d, m) 对总是产生相同的 k ——不可预测，统计均匀，且绝不重用。

3.3 签名可锻性与 Low-S 规范化

3.3.1 可锻性的数学根源

ECDSA 签名具有内在对称性： (r, s) 和 $(r, n - s)$ 对同一消息均有效。原因在于椭圆曲线上的取反操作：

$$R' = e \cdot (n - s)^{-1} \cdot G + r \cdot (n - s)^{-1} \cdot P$$

由于 $(n - s)^{-1} \equiv -s^{-1} \pmod{n}$ ：

$$R' = -e \cdot s^{-1} \cdot G - r \cdot s^{-1} \cdot P = -(e \cdot s^{-1} \cdot G + r \cdot s^{-1} \cdot P) = -R$$

点 $-R = (R_x, -R_y)$ 与 R 有相同的 x 坐标，因此验证通过。每个消息-密钥对因此存在两个不同签名。

3.3.2 比特币中的影响

攻击者可将未确认交易的 s 替换为 $n - s$ ，生成新 txid 后重新广播。这破坏了闪电网络等依赖 txid 稳定性的上层协议。比特币通过 BIP-62 → BIP-66 → BIP-146 → BIP-141 (SegWit) 的渐进式路线最终将 Low-S 规则变为共识强制要求。

3.3.3 libsecp256k1 中的实现

源码 `src/ecdsa_impl.h` 第 298–302 行，签名函数默认对 s 执行 Low-S 规范化：

```
high = secp256k1_scalar_is_high(sigs);
secp256k1_scalar_cond_negate(sigs, high);
if (recid) {
    *recid ^= high;
}
```

`secp256k1_scalar_is_high` 检查 $s > n/2$ ，`secp256k1_scalar_cond_negate` 在条件为真时将 s 替换为 $n - s$ 。

四 密码算法相关的性能优化

4.1 safegcd 模逆算法

这是 libsecp256k1 历史上最显著的单一性能优化。README 明确将其列为实现细节之一：

```
Modular inverses (both field elements and scalars) based on safegcd with
some modifications, and a variable-time variant (by Peter Dettman).
```

4.1.1 为什么模逆运算是性能瓶颈

在 ECDSA 签名过程中，模逆运算是计算开销最大的操作之一：

- **签名生成：** $s = k^{-1}(e + d \cdot r) \bmod n$ 需要计算一次标量 k 的模逆
- **签名验证：** $w = s^{-1} \bmod n$ 需要计算一次标量 s 的模逆

传统求模逆的方法需要对 256 位大整数进行数百次迭代运算，每次迭代都要更新完整的 256 位操作数。在优化之前，一次模逆运算约占签名总时间的 30%—40%，是名副其实的性能瓶颈。

4.1.2 safegcd 算法的核心创新

safegcd 算法由 Bernstein 与 Yang 于 2019 年提出。其核心创新在于将模逆计算降维到 64 位寄存器层面。

算法维护的状态变量：

- δ ：有符号整数，跟踪算法的收敛进度
- f 和 g ：两个奇数，是 GCD 计算中的核心操作数

divstep（除法步）的概念： 算法最基本的一步称为 divstep。在每一步中，算法的行为完全由 g 的最低有效位决定，涉及的运算仅是加、减、移位等简单操作，系数仅为 $(0, 2, -1, 1)$ 等小整数。

N 步批处理： 接下来的 N 步 divstep 完全只依赖于 f 和 g 的底部 N 位以及 δ 的当前值，而不需要 f 和 g 的高位信息。基于这一观察，算法将每 $N = 62$ 个 divstep 打包成一个批次进行处理：

1. **内层循环：** 用 63 位有符号整数运算完成 62 步 divstep 的迭代
2. **关键优势：** 这 62 步中完全不涉及任何 256 位大整数运算
3. **外层更新：** 每完成 62 步后，通过一个转移矩阵一次性更新完整的 256 位 f 和 g 值

这种设计将最内层循环从 256 位大整数运算降维到 64 位寄存器运算，是性能飞跃的根本原因。根据源码中的注释，转移矩阵乘法约占模逆总时间的 25%。

详细的算法阐释可见 doc/safegcd_implementation.md，该文档包含从 GCD 到模逆再到常数时间实现的完整 Python 原型代码及逐步推导。

4.1.3 收敛性： δ 的作用与迭代界

δ 的作用是引导算法向数值收敛的方向前进。每次 divstep 将较大的数替换为较小的数或零，而 δ 跟踪两个操作数之间的距离关系。当 $g = 0$ 时， $|f|$ 即为所求的最大公约数。

算法收敛的上界经过持续的数学优化：

表 1: safegcd 迭代界的持续优化

贡献者	迭代界
Bernstein & Yang (2019)	≤ 741
Pieter Wuille	≤ 724
Wuille & G. Maxwell	≤ 590
R. O'Connor (Blockstream)	$\leq 590 \checkmark$

- **常数时间版本：** 迭代 $\lceil 590/62 \rceil = 10$ 批次。多余的步数在 $g = 0$ 后不再影响结果，但执行固定步数是为了保证常数时间安全性
- **可变时间版本：** 使用早停优化，通过 count_trailing_zeros 批量跳过 g 为偶数的无效步骤，在非秘密数据场景下获得更高性能

4.2 SDMC 带符号位多梳点乘算法

The implementation of the point multiplication algorithm used for signing and public key generation was changed, resulting in improved performance for those operations.

这是 libsecp256k1 中点乘算法的全面革新，由 Peter Dettman 设计、Pieter Wuille 适配，源码位于 `src/ecmult_gen.h` 和 `src/ecmult_gen_impl.h`。

4.2.1 算法配置

算法由三个核心参数控制：

- **COMB_BLOCKS**：标量分块数，每块有独立预计算表
- **COMB_TEETH**：每次查表同时处理的位数
- **COMB_SPACING**：加倍操作次数

默认配置：COMB_BLOCKS=11，COMB_TEETH=6，对应的预计算表大小为 22 KiB。

梳齿间距 COMB_SPACING 的加倍次数仅为 $\lceil 256 / (\text{BLOCKS} \times \text{TEETH}) \rceil$ ，而非传统 double-and-add 的约 255 次。

4.2.2 标量重编码与数学推导

核心关系：

$$\text{comb}(s, P) = \sum_{i=0}^{\text{COMB_BITS}-1} (2s[i] - 1) \cdot 2^i \cdot P \quad (3)$$

展开二进表示：

$$\text{comb}(s, P) = (2 \cdot s - (2^{\text{COMB_BITS}} - 1)) \cdot P \quad (4)$$

若想计算 $(gn - b) \cdot G$ 作为 $\text{comb}(d, G/2)$ ，标量需满足：

$$d = gn - b + \frac{2^{\text{COMB_BITS}} - 1}{2} \pmod{n} \quad (5)$$

预计算表存储 $G/2$ 的倍数，使得标量 d 在乘法时无需除以 2。这一开销完全移入一次性预计算阶段。标量偏移量预存入 `ctx->scalar_offset`。

最终计算（源码第 109–110 行）：

```
/* Compute the scalar d = (gn + ctx->scalar_offset). */
secp256k1_scalar_add(&d, &ctx->scalar_offset, gn);
```

然后对重编码后的标量执行梳齿迭代，末尾加上 `ctx->ge_offset` 来校正盲化。

4.2.3 盲化与安全设计

盲化策略：

$$R = (gn - b) \cdot G + b \cdot G \quad (6)$$

其中 b 是随机盲化标量， $b \cdot G$ 预存入 `ctx->ge_offset`。这使得攻击者无法控制实际参与梳齿计算的标量值，即使控制了签名输入的 gn 。

4.2.4 源码中的时间侧信道防护细节

SDMC 的查表过程使用条件移动而非直接数组索引：

```
for (index = 0; index < COMB_POINTS; ++index) {
    secp256k1_ge_storage_cmov(&adds,
        &secp256k1_ecmult_gen_prec_table[block][index],
        index == abs);
```

遍历所有表条目，仅当 `index == abs` 时实际移动数据。这消除了秘密依赖的内存访问模式。

4.3 128 位宽乘法与 Montgomery 模乘

这是 libsecp256k1 在底层算术运算上的一项重要优化。CHANGELOG v0.2.0 记载：

Added support for 128-bit wide multiplication on MSVC for x86_64 and arm64, giving roughly a 20% speedup on those platforms.

4.3.1 数学需求：为什么需要 128 位乘法

secp256k1 的基域 $\mathbb{F}_p$ 中，模乘运算 $a \times b \bmod p$ 需要计算两个 256 位整数的乘积，中间结果高达 512 位。libsecp256k1 使用 Montgomery 乘法进行高效的模乘运算，其核心思想是将模除运算转化为移位操作。

Montgomery 乘法计算的是 $a \times b \times R^{-1} \bmod p$ ，其中 $R = 2^{256}$ 。算法步骤如下：

1. $T \leftarrow a \times b$ (512 位中间乘积，需要 128 位乘法)
2. $m \leftarrow (T \bmod R) \times p' \bmod R$ ，其中 $p' = -p^{-1} \bmod R$ 为预计算常数
3. $t \leftarrow (T + m \times p) / R$ (除以 $R = 2^{256}$ 即为右移 256 位，无需除法指令)
4. 若 $t \geq p$ 则 $t \leftarrow t - p$

步骤 1 和步骤 3 中，512 位的乘积累加操作是 128 位乘法发挥作用的关键场所。

4.3.2 编译器差异与 MSVC 的挑战

GCC 和 Clang 编译器提供了 `__int128` 扩展类型，能够自动将 128 位运算映射到 CPU 的原生指令：

- x86_64 平台：映射到 `mulx` 指令
- ARM64 平台：映射到 `umulh` 指令

然而，MSVC (Microsoft Visual C++) 编译器不支持 `__int128`。在此之前，MSVC 用户只能使用低效的软件模拟实现，并处理复杂的进位传播逻辑，性能损失严重。

4.3.3 修复方案：MSVC 内置函数

v0.2.0 通过 MSVC 提供的编译器内置函数 (intrinsics) 解决了这一问题：

- `_umul128`: x86_64 平台上的 128 位无符号乘法
- `_mul128`: x86_64 平台上的 128 位有符号乘法
- ARM64 平台: MSVC 也提供了相应的 128 位乘法内置支持

源码中提供了三种 `int128` 实现选择, 构建系统通过特性检测自动选择最优路径:

1. `int128_native`: GCC/Clang 的 `__int128` (最快)
2. `int128_struct`: MSVC 内置函数支持的 128 位结构体
3. 通用 limb 分解方式 (性能最差, 作为备选回退方案)

4.3.4 性能提升

启用原生 128 位乘法后, MSVC 平台上的模乘运算性能提升约 **20%**。这一优化不仅惠及 Windows 开发者, 也使得在 ARM64 架构上使用 MSVC 编译的代码能够获得与 GCC/Clang 相当的性能水平。

该优化是 `libsecp256k1` 在保持跨平台可移植性的同时, 针对不同编译器特性进行精细化性能调优的典型示例。

4.4 ECDH 点乘算法替换与 x86_64 汇编移除

CHANGELOG v0.4.1 记载了 `libsecp256k1` 在性能优化策略上的一个重要转变:

The point multiplication algorithm used for ECDH operations (module `ecdh`) was replaced with a slightly faster one.

Optional handwritten x86_64 assembly for field operations was removed because modern C compilers are able to output more efficient assembly. This change results in a significant speedup ... Benchmarks with GCC 10.5.0 show a 10% speedup for `secp256k1_ecdsa_verify` and `secp256k1_schnorrsg_verify`.

这是一个反直觉的结果: 移除手写汇编、改用纯 C 代码, 反而获得了 10% 的性能提升。其深层原因在于现代编译器的三个优势:

1. **自动向量化与指令调度**: GCC 10+ 的自动向量化和指令调度优化已经能匹敌甚至超越手写汇编。现代 CPU 的乱序执行和微架构差异使得编译器比人类更适合进行指令级优化。
2. **跨微架构兼容**: 手写汇编针对特定 CPU 微架构（如 Skylake）调优，在更新的微架构（如 Zen 4、Alder Lake）上未必最优。C 编译器针对目标平台自动调整。
3. **可审计性**: 每行汇编的语义密度远高于 C 代码，且缺乏类型检查。纯 C 的实现更易于审查和验证正确性。

4.4.1 ECDH 点乘算法的替换

ECDH 密钥交换协议需要计算共享秘密 $S = a \cdot P$ ，其中 a 是私钥， P 是对方的公钥。v0.4.1 将 ECDH 的点乘算法替换为更快的实现，具体优化包括：

1. **wNAF 编码**: 将标量 a 表示为窗口非相邻形式，减少非零数字的数量，从而减少点加操作的次数
2. **Shamir 技巧**: 在验证场景中同时计算 $a \cdot P + b \cdot G$ ，利用 Shamir 的批量乘法技巧将两个标量乘法的开销合并
3. **GLV 分解**: 利用 secp256k1 的高效可计算自同态 ϕ （满足 $\phi^2 + \phi + 1 = 0$ ），将 256 位标量 a 分解为两个 128 位的子标量 a_1 和 a_2 ，使得 $a \cdot P = a_1 \cdot P + a_2 \cdot \phi(P)$ ，从而将点加倍次数从约 255 次减少到约 128 次

4.4.2 启发

- 手写汇编的收益是静态的，而编译器的优化能力是动态提升的
- 纯 C 代码具有更好的可移植性和可审计性，符合 libsecp256k1 的安全设计原则
- 在安全攸关的密码学库中，代码的可验证性比微小的性能收益更为重要
- 这一决策也体现了 libsecp256k1 项目持续追求在保持最高安全标准的前提下，让编译器为我们做更多工作的理念。

五 数学原理总结

5.1 椭圆曲线群运算

secp256k1 曲线定义在有限素数域 $\mathbb{F}_p$ 上，方程为 Weierstrass 标准形式 $y^2 = x^3 + 7$ （即 $a = 0, b = 7$ 的特例）。椭圆曲线上的点构成一个加法交换群，其运算规则如下：

仿射坐标下的点加法 ($P \neq Q$):

$$\lambda = \frac{y_Q - y_P}{x_Q - x_P}, \quad x_R = \lambda^2 - x_P - x_Q, \quad y_R = \lambda(x_P - x_R) - y_P \quad (7)$$

仿射坐标下的点加倍 ($P = Q$):

$$\lambda = \frac{3x_P^2}{2y_P}, \quad x_R = \lambda^2 - 2x_P, \quad y_R = \lambda(x_P - x_R) - y_P \quad (8)$$

由于 $a = 0$ ，点加倍公式中的分子简化为 $3x_P^2$ ，减少了模 p 下的域乘法次数。

Jacobian 投影坐标：仿射坐标 (x, y) 对应 Jacobian 坐标 (X, Y, Z) ，满足 $x = X/Z^2, y = Y/Z^3$ 。Jacobian 坐标的优势在于点加和点加倍都不需要模 p 逆运算，仅需在最后还原为仿射坐标时进行一次模逆。

5.2 椭圆曲线离散对数问题

ECDSA 的安全性建立在椭圆曲线离散对数问题 (ECDLP) 的困难性之上：给定公钥 $P = d \cdot G$ ，求解私钥 d 。

secp256k1 的曲线阶 n 是一个 256 位素数 ($n \approx 2^{256} - 2^{128}$)。对于素数阶椭圆曲线，已知最佳求解算法是 Pollard's rho 算法，其时间复杂度为 $O(\sqrt{n}) \approx 2^{128}$ 次群运算。这一量级在经典计算模型下意味着即使使用全球所有计算资源并行攻击，所需时间也远超宇宙年龄 (10^{10} 年量级)。因此，secp256k1 提供约 128 位的安全强度，足以抵御所有已知经典（非量子）攻击。

量子计算威胁：Shor 算法可在量子计算机上以多项式时间求解 ECDLP。对于 256 位椭圆曲线，理论上需要约数千个逻辑量子比特，但目前量子计算机的规模和稳定性远未达到这一水平。

5.3 模逆与 GCD 算法的关系

模逆运算 $a^{-1} \bmod n$ 在数学上等价于求解 Bézout 等式：

$$a \cdot x + n \cdot y = 1 \quad (9)$$

当且仅当 $\gcd(a, n) = 1$ 时解存在，此时 $x \equiv a^{-1} \pmod{n}$ 。因此，模逆算法的本质就是扩展欧几里得算法（Extended Euclidean Algorithm）——通过在辗转相除的过程中跟踪系数，得到满足 Bézout 等式的 x 和 y 。

Bernstein-Yang safegcd 的数学原理：传统扩展欧几里得算法每次迭代需要处理完整的 256 位操作数，而 safegcd 通过 divstep 批处理将计算效率提升了近一个数量级。其核心数学洞察是：

1. 每个 divstep 的决策（变换矩阵的选择）仅依赖于 f 和 g 的最低位以及状态变量 δ 的符号
2. 接下来的 $N = 62$ 个 divstep 完全由 f 和 g 的底部 62 位以及 δ 的当前值决定
3. 因此，内层 62 步可以在 64 位寄存器精度下完成，完全不涉及 256 位大整数运算
4. 每 62 步后，通过转移矩阵一次性更新完整的 256 位操作数
5. 这一策略将内层循环从 256 位大整数运算降维到 64 位寄存器运算，是性能提升的根本原因。

5.4 标量乘法的代数优化空间

标量乘法 $k \cdot P$ 是椭圆曲线密码中最基本也最耗时的运算。可以从多个代数角度进行优化，本报告涉及的核心优化方法及其数学原理对比如下：

GLV 分解的数学原理：secp256k1 的端自同态 ϕ 满足 $\phi^2 + \phi + 1 = 0$ ，其作用是在不离开椭圆曲线群的前提下，将点 P 映射到点 $\phi(P)$ 。对于任意标量 k ，可以分解为 $k \equiv k_1 + k_2 \cdot \lambda \pmod{n}$ ，其中 $k_1, k_2 \approx 2^{128}$ ，从而将 $k \cdot P$ 转化为 $k_1 \cdot P + k_2 \cdot \phi(P)$ ，点加倍次数从约 255 次减少到约 128 次。

SDMC（梳齿）算法的数学原理：将标量 k 按位分组和按齿排列，预计算 $G/2$ 的倍数表。每个齿的查表贡献用条件移动实现常数时间安全，最终通过累加所有贡献得到结果。在 86 KiB 预计算表配置下，加倍次数仅为 3 次，这是用空间换时间的经典策略。

5.5 哈希函数在 ECDSA 中的不可替代性

SHA-256 在 ECDSA 安全模型中承担两个不可替代的角色，两者中任何一个失效都将导致 ECDSA 的完全崩溃：

1. **消息绑定：** $e = H(m)$ 的原像阻力阻止了攻击者构造一条消息 m' 使得 $H(m') = e'$ 。若 SHA-256 被攻破，即使 ECDLP 仍然安全，攻击者可以为任意伪造的哈希值 e' 构造有效签名，然后找到对应的合法消息。
2. **确定性 nonce 派生：**RFC 6979 中的 $k = \text{HMAC-SHA256}(d, e)$ 利用 HMAC 的伪随机性将 nonce 的随机性需求转嫁到私钥 d 的秘密性上。这一设计消除了对外部随机数生成器的信任依赖，从根本上避免了因随机源缺陷导致的 nonce 重用攻击。
3. ECDSA 的安全性是 ECDLP 困难性和 SHA-256 抗碰撞性/抗原像性的联合保证，这两者缺一不可。任何一方的突破都将导致整个签名系统失效。